

# SWRouter: Similarity-Contractive Window Routing for Multi-Turn Large Language Model Conversations

YU WANG, Department of Big Data Management and Application, Shanghai Jiao Tong University, China

YUCHEN LI, School of Computer Science, Shanghai Jiao Tong University, China and Baidu Inc., China

RUI KONG, Baidu Inc., China

XINRAN CHEN, Baidu Inc., China

JIAMIN CHEN, Baidu Inc., China

HENGYI CAI, Baidu Inc., China

SHUAIQIANG WANG, Baidu Inc., China

JIASHU ZHAO, Wilfrid Laurier University, Canada

YULUN ZHANG, School of Computer Science, Shanghai Jiao Tong University, China

ZHONGHAO LYU, Department of Computer Science, The Hong Seng University of Hong Kong, China

HAOYI XIONG, Baidu Inc., China

LINGHE KONG, School of Computer Science, Shanghai Jiao Tong University, China

JIMMY XIANGJI HUANG, York University, Canada

DAWEI YIN, Baidu Inc., China

Large language models exhibit complementary strengths, motivating routing methods that dispatch each query to the most suitable model. Although existing routers are effective in single-turn settings, they do not directly transfer to multi-turn dialogue, where routing performance critically depends on how historical context is segmented, retained, and incorporated into the current prompt. This introduces two fundamental challenges: preventing information loss and information confusion during context construction, and evaluating routing quality without conflating model selection with prompt construction quality. In this paper, we propose **SWRouter**, a Similarity-Contractive Window Router for multi-turn large language model routing. SWRouter combines a similarity-based context segmentation mechanism for prompt construction with a dual-metric evaluation framework that decouples construction accuracy from router performance. Experiments on multi-turn dialogue benchmarks demonstrate that **SWRouter** consistently surpasses strong baselines, achieving a **16.26%** improvement in evaluation accuracy over the best individual large language model and an additional **8.22%** gain over the Conv-ID Context baseline. Our results highlight that multi-turn large language model routing requires a joint design of context construction and evaluation, rather than a direct extension of single-turn routing methods.

CCS Concepts: • **Information systems** → **Retrieval models and ranking**; • **Computing methodologies** → *Natural language generation*; *Machine learning approaches*.

---

Authors' Contact Information: Yu Wang, wangyv123@sjtu.edu.cn, Department of Big Data Management and Application, Shanghai Jiao Tong University, Shanghai, China; Yuchen Li, yuchenli1230@gmail.com, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China and Baidu Inc., Beijing, China; Rui Kong, monster119120@gmail.com, Baidu Inc., Beijing, China; Xinran Chen, fantastique0910@gmail.com, Baidu Inc., Beijing, China; Jiamin Chen, jmchen26-c@my.cityu.edu.hk, Baidu Inc., Beijing, China; Hengyi Cai, hengyi1995@gmail.com, Baidu Inc., Beijing, China; Shuaiqiang Wang, shqiang.wang@gmail.com, Baidu Inc., Beijing, China; Jiashu Zhao, jzhao@wlu.ca, Wilfrid Laurier University, Waterloo, Canada; Yulun Zhang, yulun100@gmail.com, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China; Zhonghao Lyu, lzho@kth.se, Department of Computer Science, The Hong Seng University of Hong Kong, Hong Kong SAR, China; Haoyi Xiong, haoyi.xiong.fr@ieee.org, Baidu Inc., Beijing, China; Linghe Kong, linghe.kong@sjtu.edu.cn, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China; Jimmy Xiangji Huang, jhuang@yorku.ca, York University, Toronto, Canada; Dawei Yin, yindawei@acm.org, Baidu Inc., Beijing, China.

Additional Key Words and Phrases: Large Language Models, Multi-turn Dialogue, LLM Routing, Context Segmentation, Evaluation Framework, Model Selection

## 1 Introduction

Large Language Models (LLMs) have become core infrastructure for a wide range of intelligent applications, including conversational assistants, code generation, content creation, and increasingly, AI search and agentic question answering [10, 27, 28]. Their rapid adoption has also stimulated growing efforts to improve the efficiency of long-context processing and inference [29, 30]. Meanwhile, many organizations have developed their own LLM families, such as OpenAI’s ChatGPT [35], DeepSeek’s DeepSeek [15], Meta’s LLaMA [42], Google’s Gemini [17], and Anthropic’s Claude [5]. However, no single model consistently performs best across all tasks and applications. For example, OpenAI’s GPT series is known for strong text generation ability [35], DeepSeek demonstrates strength in mathematical reasoning [39], and Claude is particularly effective in code generation and understanding [5]. Such heterogeneity has motivated a growing line of work on *LLM routing*, which dynamically dispatches each query to the most suitable model rather than relying on a fixed model for all inputs.

Existing routing methods have achieved promising results in single-turn settings, where the routing decision is made from a single prompt in isolation. Representative examples include routers that select among models of similar scale, such as RouterDC [9], as well as methods that decide between small and large models, such as C2MAB-V [14]. However, directly applying these routing strategies to multi-turn dialogue is fundamentally difficult, because model selection in conversational systems depends not only on the current user query, but also on how the dialogue history is segmented, retained, and incorporated into the prompt. In multi-turn dialogue, naive use of historical context introduces two fundamental challenges.

- First, there is a **lack of context segmentation mechanism**. When historical information is not properly segmented and integrated, the router may fail to distinguish relevant from irrelevant past context. This leads to two concrete failure modes: **information loss**, where critical historical information required for the current response is omitted, and **information confusion**, where outdated context is mistakenly carried into a new topic. As illustrated in Figure 1, missing prior context can cause failure on rule-dependent queries, while incorrect reuse of obsolete context can degrade performance after topic shifts.
- Second, there is an **attribution bias in multi-turn router evaluation**. In multi-turn settings, final response quality is jointly determined by two factors: the quality of context construction and the selected model’s performance under the constructed prompt. A poorly constructed prompt may substantially degrade the responses of all candidate models, regardless of which model is selected. Conversely, even with well-constructed context, poor model selection can still lead to unsatisfactory responses.

As illustrated in Figure 1, a well-combined prompt and a poorly combined prompt may lead to dramatically different response quality across all candidate models (e.g., 0.9/0.7/0.3 vs. 0.09/0.07/0.03). However, the observed inference accuracy alone does not indicate whether such performance variation comes from the context construction stage or from errors in the trained router. This conflates construction quality with routing quality, making faithful evaluation of multi-turn routing systems difficult.

To address these issues, we propose **SWRouter**, a multi-turn LLM routing *framework* that jointly addresses context construction, router training, and evaluation. SWRouter consists of three tightly coupled components: (1) a **similarity-based context segmentation** mechanism that selectively incorporates semantically relevant historical turns when

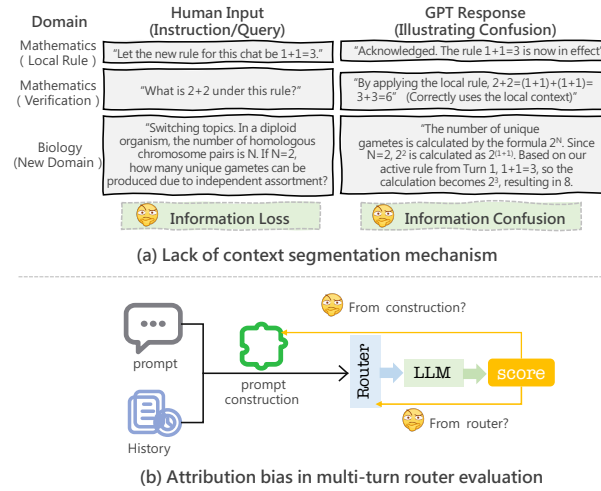


Fig. 1. Illustration of the two challenges in multi-turn large language model routing. **(a)** The context-construction stage receives a dialogue history and the current user turn, then decides which historical turns should be retained before routing. When this step drops necessary history, the routed model receives an under-specified prompt and suffers from **information loss**; when it carries obsolete turns across a topic shift, the prompt contains misleading context and causes **information confusion**. **(b)** The evaluation stage observes only the final routed response score, but that score is jointly determined by prompt construction and model selection. A high-quality window can raise the absolute scores of all candidate LLMs, whereas a poor window can depress them even if the router still chooses the relatively best model, producing attribution bias unless construction quality and router performance are measured separately.

constructing the prompt, thereby reducing both information loss and information confusion; (2) a **pluggable router backbone** trained on the constructed prompts with contrastive objectives to select the most suitable model; and (3) a **decoupled evaluation framework** that separates *construction accuracy* from *router performance*, enabling faithful assessment of each component independently. We evaluate SWRouter on multi-turn dialogue benchmarks and compare it against representative routing baselines and strong individual LLMs. Empirically, **SWRouter** consistently matches or surpasses the Conv-ID Context baseline, improves average accuracy by **16.26%** over the best individual LLM, and further achieves an **8.22%** gain over the Conv-ID Context baseline on multi-turn dialogue benchmarks.

Our contributions are summarized as follows:

- We identify that the lack of context segmentation in multi-turn dialogue routing leads to two concrete failure modes—**information loss** and **information confusion**. To address this issue, we propose a similarity-based prompt construction mechanism that integrates historical and current information.
- We reveal a classification bias in existing router evaluation protocols and introduce a decoupled evaluation framework with three complementary metrics: **evaluation accuracy**, **construction accuracy**, and **router performance**.
- We construct a strong Conv-ID Context baseline and conduct extensive in-distribution and out-of-distribution evaluations. **SWRouter** achieves **16.26%** and **8.22%** gains over the best single LLM and Conv-ID Context on multi-turn benchmarks, and outperforms Conv-ID Context by **2.68%** on OOD tasks. Decoupled metric analysis further confirms that the gains stem from improved **construction accuracy** ( $\bar{w}$ ), while **router performance** ( $P_{\text{router}} > 1$ ) validates effective model selection across all settings.

## 2 Background and Motivation

### 2.1 Problem Formulation

In conventional single-turn LLM routing, the routing decision is made from the current prompt alone: given a user query, the router selects the candidate model that is expected to produce the best response. In multi-turn dialogue, however, this problem becomes fundamentally more challenging. The router must not only determine *which* model to select, but also decide *what historical context should be preserved* and *how that context should be incorporated* into the current prompt.

Formally, let  $\mathcal{M} = \{M_t\}_{t=1}^T$  denote a pool of  $T$  candidate LLMs, and let  $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$  denote the training set, where  $x_i$  is the current user input and  $y_i$  is the corresponding target answer. In a single-turn setting, routing is performed directly on  $x_i$ . In a multi-turn setting, by contrast, the current turn should be interpreted together with relevant dialogue history. Therefore, the effective routing input is no longer the raw query  $x_i$ , but a *context-enhanced query*  $c_i$ , which is constructed by combining the current turn with selected historical information.

More specifically, for each current query  $x_i$ , we construct a combined query  $c_i$  by integrating it with semantically relevant historical turns. This yields a transformed training set

$$\mathcal{C}_{\text{train}} = \{(c_i, y_i)\}_{i=1}^n,$$

where  $c_i$  captures the multi-turn context on which routing should be based. The router then produces a probability distribution over the candidate model pool  $\mathcal{M}$  and selects the model that is most suitable for answering under the constructed prompt.

Frequently used symbols are summarized in Table 1.

This formulation highlights a key distinction between single-turn and multi-turn routing: in multi-turn dialogue, routing quality depends not only on model selection, but also on prompt construction. If relevant history is omitted, the routed model may fail because of *information loss*; if irrelevant or outdated history is retained, the routed model may instead suffer from *information confusion*. As illustrated in Figure 1, these two failure modes arise before the routing decision itself and directly affect the quality of downstream responses.

### 2.2 Single-turn LLM Routing and Its Limitation

Among existing single-turn routing methods, RouterDC [9] is a representative approach. It learns a router that maps an input query to a probability distribution over candidate LLMs, and optimizes the routing process with dual contrastive objectives. This design is effective in single-turn scenarios because the router only needs to reason over one prompt in isolation [9].

However, this assumption does not hold in multi-turn dialogue. A single-turn router such as RouterDC [9] takes the current prompt as the routing unit, while ignoring the fact that the prompt itself may be under-specified or even misleading if dialogue history is not properly segmented and incorporated. As a consequence, even when the router selects the relatively best model for the observed input, the end-to-end system can still fail because the routing input is poorly constructed.

This limitation is particularly important in our setting for two reasons. First, multi-turn dialogue introduces a *context selection problem*: the system must decide which historical turns are relevant to the current request before routing can be performed reliably. Second, conventional router evaluation is largely based on relative ranking signals, which are suitable for stable optimization but insufficient for measuring end-to-end usefulness when prompt construction quality

Table 1. Symbols and Definitions.

| Symbol                          | Definition                                                                                 |
|---------------------------------|--------------------------------------------------------------------------------------------|
| $\mathcal{M} = \{M_t\}_{t=1}^T$ | Pool of $T$ candidate LLMs available for routing.                                          |
| $x_i$                           | Current user query or turn before context construction.                                    |
| $y_i$                           | Target answer or supervision associated with $x_i$ .                                       |
| $H_i$                           | Historical window retained for the $i$ -th query after similarity-based segmentation.      |
| $c_i$                           | Context-enhanced query obtained by merging $x_i$ with the selected historical window.      |
| $\mathcal{D}_{\text{train}}$    | Original training set before window-based prompt construction.                             |
| $\mathcal{C}_{\text{train}}$    | Transformed training set composed of context-enhanced queries.                             |
| $E(\cdot; \mathbf{w})$          | Encoder that maps a query or constructed prompt into a dense representation.               |
| $e_i$                           | Embedding of $x_i$ produced by $E(\cdot; \mathbf{w})$ .                                    |
| $\text{sim}(\cdot, \cdot)$      | Cosine similarity used for window partitioning and model matching.                         |
| $\tau$                          | Similarity threshold controlling whether adjacent turns are merged or split.               |
| $k_t$                           | Learnable embedding representing candidate model $M_t$ .                                   |
| $R(c_i; \theta)$                | Router distribution over candidate LLMs for constructed prompt $c_i$ .                     |
| $I_i^+, I_i^-$                  | Positive and negative candidate model sets used by the sample-LLM contrastive loss.        |
| $L_{\text{sample-LLM}}$         | Contrastive loss that separates top-performing and bottom-performing LLMs for each query.  |
| $L_{\text{sample-sample}}$      | Contrastive loss that aligns semantically related constructed prompts.                     |
| $w_{i,t}$                       | True score of model $M_t$ on constructed prompt $c_i$ , normalized to $[0, 1]$ .           |
| $s_i^{(t)}$                     | Binary indicator showing whether the router selects model $M_t$ for $c_i$ .                |
| $\bar{s}$                       | Evaluation accuracy, i.e., the average true score of routed responses.                     |
| $\bar{w}$                       | Construction accuracy, i.e., the average true score over all candidate models and prompts. |
| $P_{\text{router}}$             | Router performance, defined as $\bar{s}\bar{w}$ .                                          |

varies. As discussed in Section 4.4, two prompts can induce the same relative ranking over candidate models while yielding drastically different true scores. Therefore, directly extending a single-turn router to multi-turn dialogue is insufficient.

These observations motivate a routing framework that jointly addresses *context construction* and *routing evaluation*. In the next section, we introduce SWRouter, which augments router learning with similarity-based window partitioning for prompt construction and a decoupled evaluation framework for faithful assessment in multi-turn settings.

### 3 Preliminaries and Problem Setup

This section formalizes the multi-turn routing setting studied in this paper. The goal is to make explicit the objects that are optimized by the router and the objects that are produced by the context-construction module. Unless otherwise specified, the notation follows Table 1.

#### 3.1 Multi-turn Dialogue Instances

We consider a collection of multi-turn dialogues. For a routing instance indexed by  $i$ , let

$$\mathcal{Z}_i = \{(u_{i,1}, a_{i,1}), (u_{i,2}, a_{i,2}), \dots, (u_{i,m_i-1}, a_{i,m_i-1}), u_{i,m_i}\}$$

denote the dialogue observed before producing the next assistant response. Here,  $u_{i,j}$  is the  $j$ -th user turn,  $a_{i,j}$  is the corresponding assistant turn, and  $u_{i,m_i}$  is the current user request. For consistency with the rest of the paper, we write

$x_i = u_{i,m_i}$  for the current query. The raw dialogue history before  $x_i$  is denoted by

$$\mathcal{H}_i^{\text{raw}} = \{(u_{i,1}, a_{i,1}), \dots, (u_{i,m_i-1}, a_{i,m_i-1})\}.$$

The router does not directly operate on  $\mathcal{H}_i^{\text{raw}}$ . Instead, a context-construction function  $g(\cdot)$  first selects and organizes the relevant historical information, producing a retained semantic window  $H_i$  and a context-enhanced prompt

$$c_i = g(\mathcal{H}_i^{\text{raw}}, x_i). \quad (1)$$

In this paper,  $g(\cdot)$  is instantiated by similarity-based window construction, but the problem setup allows other context selectors such as full-history concatenation, current-turn-only prompting, or retrieval-based context selection.

This distinction is important because  $x_i$  alone may be under-specified. For example, the current request may contain pronouns, omitted constraints, or references to entities introduced in previous turns. At the same time, the entire raw history may contain outdated goals, corrected assumptions, or off-topic turns. Therefore, a valid multi-turn routing instance is not merely a user query; it is the pair consisting of the current request and the constructed prompt that determines what the candidate LLMs actually observe.

### 3.2 Candidate Models and Router Objective

Let  $\mathcal{M} = \{M_t\}_{t=1}^T$  be a fixed pool of  $T$  candidate LLMs. Given a context-enhanced prompt  $c_i$ , a router with parameters  $\theta$  outputs a probability distribution

$$R(c_i; \theta) = [R_1(c_i; \theta), R_2(c_i; \theta), \dots, R_T(c_i; \theta)],$$

where  $R_t(c_i; \theta)$  is the probability of selecting candidate model  $M_t$ . At inference time, the selected model index is

$$\hat{t}_i = \arg \max_{t \in \{1, \dots, T\}} R_t(c_i; \theta), \quad (2)$$

and the final response is generated by  $M_{\hat{t}_i}(c_i)$ .

The ideal routing decision depends on the response quality that each candidate model would obtain under the same constructed prompt. Let  $w_{i,t} \in [0, 1]$  denote the normalized quality score of model  $M_t$  on prompt  $c_i$ . The oracle candidate for this prompt is

$$t_i^* = \arg \max_{t \in \{1, \dots, T\}} w_{i,t}. \quad (3)$$

The routing objective is therefore to learn a model selector that approaches the oracle candidate while using only the constructed prompt at inference time. Equivalently, the end-to-end objective can be written as

$$\max_{\theta, g} \frac{1}{n} \sum_{i=1}^n w_{i, \hat{t}_i}, \quad \hat{t}_i = \arg \max_t R_t(g(\mathcal{H}_i^{\text{raw}}, x_i); \theta). \quad (4)$$

This expression makes explicit that multi-turn routing has two coupled components: the context-construction function  $g(\cdot)$  and the router  $R(\cdot; \theta)$ . If  $g(\cdot)$  removes necessary history, even a strong router may select a model using an incomplete prompt. If  $g(\cdot)$  introduces irrelevant history, the candidate models may all produce lower-quality responses, making the routing decision less meaningful.

### 3.3 Training Supervision and Evaluation Scores

During training and evaluation, each constructed prompt  $c_i$  is paired with responses from all candidate models. A judge model then assigns an integer score to each response, which is normalized to obtain  $w_{i,t}$ . These scores serve two

roles. First, they provide relative supervision for router learning by identifying strong and weak candidate models for each prompt. For example, the top-scoring models form the positive set  $I_i^+$ , while the bottom-scoring models form the negative set  $I_i^-$ . Second, they provide absolute response-quality measurements for decoupled evaluation.

The distinction between relative and absolute signals is central to our setting. Relative supervision is useful for training because it tells the router which candidate models should be preferred for a particular prompt. However, relative ranking alone cannot determine whether the constructed prompt itself is good. If all candidate models receive low absolute scores because the prompt omits necessary history, the best candidate model may still be easy to identify, but the final user-facing answer remains poor. For this reason, Section 4.4 separately measures evaluation accuracy, construction accuracy, and router performance.

### 3.4 Scope

We focus on routing among a fixed pool of already available LLMs. The router does not modify the candidate models, and the context-construction module operates before generation. This scope matches practical deployments in which different models have complementary strengths and costs, but invoking all models for every user request is undesirable. The problem studied here is therefore not how to train a stronger LLM, but how to construct an informative multi-turn prompt and select an appropriate model for that prompt.

## 4 Methodology

### 4.1 Overview

SWRouter is designed to address the two central challenges of multi-turn LLM routing introduced in Section 1: (1) how to construct an effective routing prompt from dialogue history, and (2) how to evaluate routing quality without conflating it with prompt construction quality. To this end, SWRouter consists of three stages: (1) **similarity-based window partitioning** to construct combined prompts from multi-turn dialogue history (Section 4.2); (2) **model routing based on the constructed prompts**, optimized with contrastive training objectives that rely on relative, ranking-based scores (Section 4.3); and (3) **decoupled evaluation** that distinguishes *construction accuracy* from *router performance* using absolute true scores after prompt construction (Section 4.4).

In the first stage, SWRouter organizes multi-turn dialogue history into semantically coherent windows. Instead of directly concatenating the entire dialogue history, it measures the semantic similarity between adjacent user turns and dynamically decides whether the current turn should be merged into the existing window or begin a new one. This design allows SWRouter to preserve relevant historical information while reducing the risk of introducing irrelevant or outdated context.

In the second stage, the constructed context-enhanced prompt is fed into a pluggable router backbone that selects the most suitable model from the candidate LLM pool. Concretely, the router backbone encodes the combined prompt into a dense representation and computes its similarity with a set of learnable model embeddings, yielding a probability distribution over candidate LLMs. The router backbone is optimized with contrastive objectives that encourage it to distinguish strong candidate models from weak ones under multi-turn settings.

In the third stage, SWRouter adopts a decoupled evaluation framework tailored to multi-turn routing. Since existing routing evaluations are typically based on relative ranking signals, they are suitable for optimization but insufficient for measuring end-to-end usefulness when prompt construction quality varies. Therefore, SWRouter separately reports

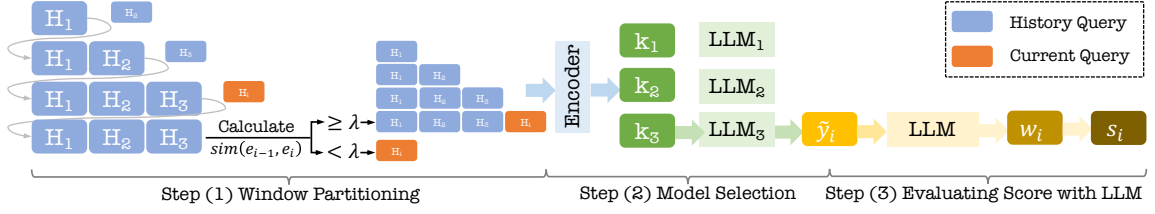


Fig. 2. Overview of SWRouter . Given a multi-turn dialogue, SWRouter first encodes adjacent user turns and computes their semantic similarity to decide whether the current turn should extend the existing window or start a new one. The retained window is then merged into a context-enhanced prompt and passed to a pluggable router, which represents the prompt with an encoder, compares it with learnable candidate-model embeddings, and selects the LLM with the highest routing score. During training, the router is optimized with sample-LLM and sample-sample contrastive objectives so that high-quality models and semantically related prompts are pulled closer in representation space. During evaluation, the generated responses are scored by judge models and decomposed into construction accuracy, evaluation accuracy, and router performance, allowing prompt construction quality and model-selection ability to be analyzed separately.

**construction accuracy**, which reflects the average quality of all candidate models under the constructed prompt, and **router performance**, which measures how much the router improves over random selection.

Overall, these three stages form a unified framework for multi-turn LLM routing. The similarity-based window partitioning stage determines what historical context should be preserved, the routing stage determines which model should be selected under the constructed prompt, and the decoupled evaluation stage determines how the resulting system should be assessed.

## 4.2 Similarity-based Window Partitioning

A core challenge in multi-turn LLM routing is that the current user query should not be interpreted in isolation. Instead, the router should determine which historical turns remain semantically relevant to the current request and should therefore be retained in the routing prompt. To address this issue, SWRouter employs a similarity-based window partitioning algorithm that organizes dialogue history into semantically coherent windows. This formulation follows the broader view of sequential structure analysis and segmentation in formal-language and string-processing algorithms [1, 7, 19], while using neural semantic similarity rather than symbolic parsing rules.

Let  $E(x; w)$  denote an encoder that maps an input utterance  $x$  into an embedding in  $\mathbb{R}^P$ . Given the current user query  $x_i$ , we first compute its embedding

$$e_i = E(x_i; w).$$

We then measure the similarity between the current query and the previous query:

$$\text{similarity} = \text{sim}(e_i, e_{i-1}), \quad (5)$$

where  $\text{sim}(\cdot, \cdot)$  denotes cosine similarity,  $e_i$  is the embedding of the current query  $x_i$ , and  $e_{i-1}$  is the embedding of the previous query  $x_{i-1}$ .

Based on this similarity score, SWRouter updates the historical window  $H_i$  as follows:

$$H_i = \begin{cases} \{x_i\}, & \text{if } \text{sim}(e_i, e_{i-1}) < \tau, \\ H_{i-1} \cup \{x_i\}, & \text{if } \text{sim}(e_i, e_{i-1}) \geq \tau, \end{cases} \quad (6)$$



---

**Algorithm 1:** Similarity-based window construction for multi-turn routing. The algorithm scans a dialogue from left to right, starts a new semantic window when adjacent user turns are weakly related, and otherwise extends the current window. For each current request, it outputs a context-enhanced prompt that contains only the active semantic window.

---

**Input:** Dialogue  $\mathcal{Z}_i = \{(u_{i,1}, a_{i,1}), \dots, (u_{i,m_i-1}, a_{i,m_i-1}), u_{i,m_i}\}$ ; encoder  $E(\cdot; \mathbf{w})$ ; threshold  $\tau$   
**Output:** Context-enhanced prompt  $c_i$  and retained window  $H_i$  for the current request  $x_i = u_{i,m_i}$

```

1  $H \leftarrow \{u_{i,1}, a_{i,1}\};$ 
2  $e_{\text{prev}} \leftarrow E(u_{i,1}; \mathbf{w});$ 
3 for  $j = 2$  to  $m_i$  do
4    $e_j \leftarrow E(u_{i,j}; \mathbf{w});$ 
5    $\rho_j \leftarrow \text{sim}(e_j, e_{\text{prev}});$ 
6   if  $\rho_j < \tau$  then
7      $H \leftarrow \{u_{i,j}\};$                                      // topic shift; reset the semantic window
8   else
9      $H \leftarrow H \cup \{u_{i,j}\};$                                // same local context; extend the window
10  if  $j < m_i$  and  $u_{i,j} \in H$  then
11     $H \leftarrow H \cup \{a_{i,j}\};$                                // retain assistant-side context inside the window
12   $e_{\text{prev}} \leftarrow e_j;$ 
13  $H_i \leftarrow H;$ 
14  $c_i \leftarrow \text{FormatPrompt}(H_i, x_i);$ 
15 return  $c_i, H_i;$ 

```

---

where  $\tau$  is a similarity threshold. Intuitively, if the current turn is insufficiently similar to the previous one, SWRouter starts a new window; otherwise, it appends the current turn to the existing window. In this way, the dialogue history is partitioned into locally coherent semantic segments rather than being naively concatenated into a single undifferentiated context.

After the historical window is determined, SWRouter constructs a context-enhanced prompt  $c_i$  for routing by combining the current query with the selected dialogue history in  $H_i$ . This design enables the router to make decisions based on relevant multi-turn context, rather than relying solely on the current turn. Compared with naive history concatenation, the similarity-based windowing mechanism offers two advantages. First, it preserves relevant context needed for rule-dependent or context-dependent requests, thereby reducing *information loss*. Second, it avoids carrying obsolete or off-topic history into the new prompt, thereby reducing *information confusion*. These two effects correspond exactly to the two failure modes illustrated in Figure 1.

The similarity threshold  $\tau$  controls the granularity of context segmentation. A smaller  $\tau$  tends to merge more turns into the same window, which may preserve more context but also increases the risk of introducing irrelevant information. A larger  $\tau$  produces more aggressive segmentation, which can better isolate topic shifts but may also discard useful dependencies. Therefore,  $\tau$  governs the trade-off between context retention and context purity, and its impact will be analyzed in Section 6.5.

Algorithm 1 summarizes the similarity-window construction procedure. The procedure is applied independently to each dialogue. It processes user turns in chronological order, compares each user turn with the immediately preceding user turn, and updates the active semantic window according to the threshold  $\tau$ . Assistant turns that lie inside the retained window are preserved when constructing the final prompt, so that the generated prompt keeps both the user’s constraints and the assistant-side facts that may be needed for resolving follow-up references.

### 4.3 Routing and Training

For a query  $x_i$ , after being transformed into a combined query  $c_i$  using the similarity-based window partitioning algorithm, SWRouter generates a selection probability distribution over  $T$  candidate LLMs:

$$R(c_i; \theta) = \text{softmax} \left( \begin{bmatrix} \text{sim}(E(c_i; w), k_1) \\ \text{sim}(E(c_i; w), k_2) \\ \vdots \\ \text{sim}(E(c_i; w), k_T) \end{bmatrix} \right), \quad (7)$$

where  $\theta \equiv \{w, k_1, k_2, \dots, k_T\}$  denotes the parameters of SWRouter, and  $\text{sim}(\cdot, \cdot)$  denotes cosine similarity.

A straightforward way to train the router is to align its output distribution with a score distribution:

$$\min_{\theta} \sum_{(c_i, y_i) \in C_{\text{train}}} \text{KL} \left( R(c_i; \theta), \text{softmax}[s_i^{(1)}, \dots, s_i^{(T)}] \right), \quad (8)$$

where  $\text{KL}(\cdot, \cdot)$  denotes the Kullback–Leibler divergence [24]. This KL objective has recently been used in LLM routing [33]. However, we argue that it is not an ideal proxy for router training in our setting, because the goal of routing is to assign queries to top-performing LLMs rather than to fit the full score distribution, especially for bottom-performing models. Following the contrastive learning spirit of RouterDC [9], SWRouter adopts contrastive training objectives over the context-enhanced query.

**4.3.1 Sample-LLM Contrastive Loss.** For each combined query  $c_i$ , SWRouter constructs positive and negative LLM index sets  $I_i^+$  and  $I_i^-$ . The sample-LLM contrastive loss is defined as

$$L_{\text{sample-LLM}}(c_i, y_i; \theta) = -\log \frac{P_i^+}{P_i^+ + P_i^-}, \quad (9)$$

where

$$P_i^+ = \sum_{t \in I_i^+} e^{\text{sim}(E(c_i; w), k_t)}, \quad (10)$$

is the sum of the exponentiated similarities between the query embedding  $E(c_i; w)$  and the top- $K_+$  model embeddings, and

$$P_i^- = \sum_{t \in I_i^-} e^{\text{sim}(E(c_i; w), k_t)}, \quad (11)$$

is the corresponding sum over the bottom- $K_-$  model embeddings. Here,  $I_i^+$  and  $I_i^-$  denote the index sets of the top- $K_+$  and bottom- $K_-$  LLMs, respectively.

**4.3.2 Sample-Sample Contrastive Loss.** In addition to contrasting candidate models for each query, SWRouter leverages unsupervised clustering to group semantically related queries and constructs a sample-sample contrastive loss: This representation-learning design is related to scalable log-linear optimization and predictive structure learning across related tasks [3, 4].

$$L_{\text{sample-sample}}(c_i; \theta) = -\log \frac{Q_i^+}{Q_i^+ + Q_i^-}, \quad (12)$$

where

$$Q_i^+ = e^{\text{sim}(E(c_i; w), E(c_i^+; w))} \quad (13)$$

---

**Algorithm 2:** Router training and inference for SWRouter . The procedure first converts raw dialogues into context-enhanced prompts using Algorithm 1, then builds contrastive supervision from candidate-model scores, optimizes the router with sample-LLM and sample-sample losses, and finally selects one candidate LLM for each test prompt.

---

**Input:** Training dialogues  $\mathcal{D}_{\text{train}}$ , test dialogues  $\mathcal{D}_{\text{test}}$ , LLM pool  $\mathcal{M} = \{M_t\}_{t=1}^T$ , encoder  $E(\cdot; \mathbf{w})$ , model embeddings  $\{k_t\}_{t=1}^T$ , threshold  $\tau$ , hyperparameters  $K_+$ ,  $K_-$ ,  $N$ ,  $\lambda$ ,  $b$ ,  $\eta$

**Output:** Trained router  $R(\cdot; \theta)$  and routed responses on the test set

- 1 **Data preprocessing:**
- 2 **for**  $\mathcal{Z}_i \in \mathcal{D}_{\text{train}}$  **do**
- 3      $(c_i, H_i) \leftarrow \text{WindowConstruct}(\mathcal{Z}_i, E, \tau)$  using Algorithm 1;
- 4     **for**  $t = 1$  **to**  $T$  **do**
- 5         Generate response  $r_{i,t} \leftarrow M_t(c_i)$ ;
- 6         Score  $r_{i,t}$  with the judge model to obtain  $w_{i,t}$ ;
- 7  $C_{\text{train}} \leftarrow \{(c_i, \{w_{i,t}\}_{t=1}^T)\}$ ;
- 8 **Training:**
- 9 Cluster queries  $\{c_i\}_{i=1}^n$  into  $N$  groups
- 10 **repeat**
- 11     Sample mini-batch  $B$  from  $C_{\text{train}}$
- 12     **for**  $(c_i, y_i) \in B$  **do**
- 13         Construct  $I_i^+$  and  $I_i^-$  using  $w_{i,t}$
- 14         Compute  $L_{\text{sample-LLM}}(c_i, y_i; \theta)$
- 15         Sample in-group query  $c_i^+$  and out-group queries  $\{c_i^-\}$
- 16         Compute  $L_{\text{sample-sample}}(c_i; \theta)$
- 17      $L(B; \theta) \leftarrow \sum_{(c_i, y_i) \in B} [L_{\text{sample-LLM}} + \lambda L_{\text{sample-sample}}]$
- 18      $\theta \leftarrow \theta - \eta \nabla_{\theta} L(B; \theta)$
- 19 **until converged;**
- 20 **Inference:**
- 21 **for**  $\mathcal{Z}_i \in \mathcal{D}_{\text{test}}$  **do**
- 22      $(c_i, H_i) \leftarrow \text{WindowConstruct}(\mathcal{Z}_i, E, \tau)$ ;
- 23      $\hat{t}_i \leftarrow \arg \max_t R_t(c_i; \theta)$ ;
- 24      $\hat{y}_i \leftarrow M_{\hat{t}_i}(c_i)$ ;
- 25 **return**  $R(\cdot; \theta)$  and  $\{\hat{y}_i\}$ ;

---

represents the similarity between the embedding of query  $c_i$  and that of a randomly chosen in-group query  $c_i^+$ , and

$$Q_i^- = \sum_{c_i^- \in X_i^-} e^{\text{sim}(E(c_i; \mathbf{w}), E(c_i^-; \mathbf{w}))} \quad (14)$$

denotes the summed similarities between  $c_i$  and a set of out-group queries  $X_i^-$ .

The overall training objective combines the above two contrastive losses:

$$\min_{\theta} \sum (\alpha L_{\text{sample-LLM}} + \beta L_{\text{sample-sample}}), \quad (15)$$

where  $\alpha$  and  $\beta$  are hyperparameters that balance the two loss terms.

Algorithm 2 summarizes the training and inference pipeline of SWRouter . The algorithm uses Algorithm 1 as a preprocessing step to transform raw multi-turn dialogues into context-enhanced prompts. Training then relies on model-level and sample-level contrastive supervision, while inference only requires a single router forward pass before calling the selected candidate LLM.

#### 4.4 Decoupled Evaluation Metrics

For each combined query  $c_i$  and each candidate model  $\mathcal{M}_t$  ( $t = 1, \dots, T$ ), we obtain a *true score*  $0 \leq w_{i,t} \leq 1$  from judge models, which reflects the absolute response quality of  $\mathcal{M}_t$  on  $c_i$ .

*Evaluation Accuracy (absolute quality).* Given the router's selection indicator  $s_i^{(t)} \in \{0, 1\}$ , where  $s_i^{(t)} = 1$  if the router selects  $\mathcal{M}_t$  for  $c_i$  and 0 otherwise, the score of the selected model is

$$s_i = \sum_{t=1}^T w_{i,t} \cdot s_i^{(t)}. \quad (16)$$

We define *evaluation accuracy* as the average true score achieved by the routing system:

$$\bar{s}_i = \frac{1}{n} \sum_{i=1}^n s_i, \quad (17)$$

which reflects the absolute usefulness of the routed responses under the constructed prompts.

*Construction Accuracy (prompt quality).* To characterize the overall quality of the constructed prompt across all candidate models, we define the average true score

$$\bar{w} = \frac{1}{nT} \sum_{i=1}^n \sum_{t=1}^T w_{i,t}, \quad (18)$$

which reflects the average capability of the candidate model pool under a given prompt construction. A higher  $\bar{w}$  indicates that the constructed prompt enables better responses across all candidate models.

*Router Performance (relative improvement).* Based on  $\bar{s}_i$  and  $\bar{w}$ , we define router performance as

$$P_{\text{router}} = \frac{\bar{s}_i}{\bar{w}}. \quad (19)$$

Here,  $\bar{w}$  reflects the average capability of the candidate model pool under a given prompt construction, while  $P_{\text{router}}$  measures how much the router improves over this average by selecting stronger models. A value greater than 1 indicates that the router outperforms random model selection.

*Discussion: three complementary metrics.* For evaluation in multi-turn dialogue, we adopt three complementary metrics to characterize the system: *evaluation accuracy* ( $\bar{s}_i$ ), *construction accuracy* ( $\bar{w}$ ), and *router performance* ( $P_{\text{router}}$ ). Figure 3 illustrates why these metrics are complementary. Under good prompt construction, the candidate models obtain scores of  $\{0.9, 0.7, 0.3\}$ , yielding an average construction accuracy of  $\bar{w} = 0.633$ . If the router selects the best model, then  $\bar{s}_i = 0.9$  and  $P_{\text{router}} = 0.9/0.633 = 1.42$ , indicating both strong construction quality and effective routing. In contrast, under poor prompt construction, information loss uniformly degrades the candidate scores to  $\{0.09, 0.07, 0.03\}$ , reducing construction accuracy to  $\bar{w} = 0.0633$ . Even if the router still selects the best model, the final evaluation accuracy drops to  $\bar{s}_i = 0.09$ , while the router performance remains unchanged:  $P_{\text{router}} = 0.09/0.0633 = 1.42$ . This example shows that high router performance does not necessarily imply high end-to-end response quality. Instead,  $\bar{s}_i$  reflects the final usefulness of the routed response,  $\bar{w}$  measures the intrinsic quality of prompt construction, and  $P_{\text{router}}$  quantifies the router's ability to select better-than-average candidates. Together, these metrics enable a decoupled analysis of construction quality and routing effectiveness.

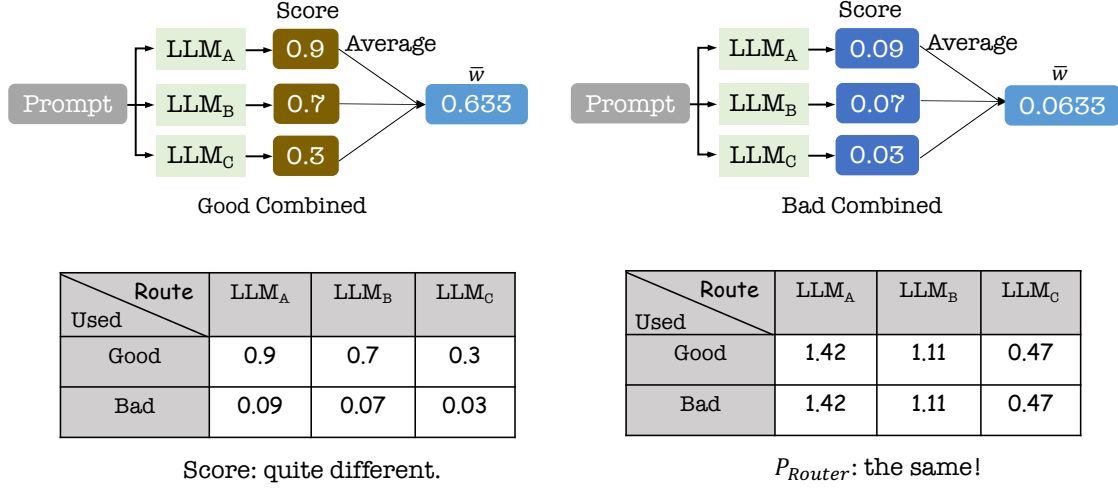


Fig. 3. Illustration of the three complementary metrics used for decoupled evaluation. For each constructed prompt, all candidate LLMs are first evaluated with absolute true scores. Construction accuracy  $\bar{w}$  averages these scores across the whole candidate pool and therefore measures whether the prompt construction stage preserves useful context for models in general. Evaluation accuracy  $\bar{s}$  then records the true score of the single model selected by the router, representing the end-to-end quality observed by users. Router performance  $P_{router} = \bar{s}/\bar{w}$  normalizes the selected-model score by the candidate average, showing whether the router improves over random model selection under the same constructed prompts.

## 5 Implementation

We implement SWRouter following the standard router-training and evaluation pipeline for multi-turn LLM routing. Our implementation consists of candidate model construction, multi-turn dataset preprocessing, similarity-window segmentation, response scoring, and router optimization.

**Candidate LLMs.** We evaluate SWRouter on seven open-source LLMs from HuggingFace: Mistral-7B [22], MetaMath-Mistral-7B [46], zephyr-7b-beta [44], Chinese-Mistral-7B [49], dolphin-2.6-mistral-7b [12], Llama-3-8B [31], and dolphin-2.9-llama3-8b [13]. The first five models are Mistral-based, while the last two are Llama-3-based. These models cover general-purpose, instruction-tuned, domain-tuned, and language-adapted variants, enabling us to evaluate whether SWRouter can select among LLMs with heterogeneous capabilities. The Llama-family candidates build on the broader open-foundation-model line represented by LLaMA and LLaMA 2 [42, 43], and zephyr-7b-beta follows preference-optimization techniques such as direct preference optimization [36].

**Datasets.** We conduct experiments on two multi-turn dialogue datasets, MTBench [6] and ShareGPT [38]. MTBench contains multi-turn conversations across diverse task categories, while ShareGPT consists of real user-AI interactions. We preprocess ShareGPT by removing noisy samples and filtering out non-multi-turn dialogues. For both datasets, we randomly split the data into 70% for training and 30% for testing. All training samples are combined as  $C_{train}$  for router training.

**Baselines.** We compare SWRouter with three groups of baselines. First, **Single Model** directly uses one candidate LLM to answer each query without routing. This group includes all seven candidate LLMs listed above. Second, **Conv-ID Context** preserves the original dialogue segmentation according to conversation IDs and performs routing with the full original multi-turn context. This baseline serves as an ID-based static context reference. Third, **ZOOTER** [33] is

used as a strong single-turn router backbone. To evaluate the generality of the proposed similarity-window mechanism, we combine ZOETER with the same similarity-window segmentation and compare it with SWRouter .

**Evaluation protocol.** We use the Language Model Evaluation Harness [16] for model evaluation, following common benchmark practice from MMLU-style multitask evaluation [21]. For open-ended generation, we generate  $M = 10$  candidate responses using stochastic beam search, where beam expansion is sampled with temperature 0.2. The generated responses are then scored by judge models, and the router is evaluated using testing weighted accuracy.

**Similarity-window configurations.** We use mDeBERTaV3-base [20] as the encoder  $E(x; w)$ , which contains approximately 86M backbone parameters. This choice follows the use of efficient neural encoders for structured text analysis [37]. The similarity threshold is set to  $\tau = 0.91$  for constructing similarity windows. The LLM embedding dimension is 768, and the number of clusters is set to  $N = 5$ .

**Router training configurations.** The router is trained for 1,000 steps using AdamW [32]. The learning rate is  $5 \times 10^{-5}$ , the weight decay is 0.01, and the batch size is 16. All experiments are conducted on NVIDIA A100 80GB GPUs.

## 6 Evaluation

We evaluate SWRouter to answer the following questions:

- (1) Can SWRouter improve routing performance on multi-turn dialogue tasks?
- (2) How well does SWRouter generalize to out-of-distribution (OOD) scenarios?
- (3) How much does each component contribute to the effectiveness of SWRouter ?
- (4) How sensitive is SWRouter to key hyperparameters?
- (5) What is the computational overhead introduced by SWRouter ?

### 6.1 Evaluation Setup

**6.1.1 Prompts Used for Evaluation.** To evaluate the quality of routed responses in multi-turn dialogue, we employ a judge model with the following evaluation prompt: The prompt is written as a structured evaluation rubric following standard academic reporting conventions [2].

*You are given a multi-turn conversation, including the previous dialogue history, the current user request, and an assistant response generated by a candidate LLM. Please evaluate the quality of the assistant response with a single integer score from 1 to 10. Judge the response semantically rather than by exact wording: paraphrases, different ordering, or different surface expressions should be treated as acceptable when they preserve the same meaning and satisfy the user’s request. Do not reward verbosity by itself; longer answers should receive higher scores only when the additional content improves correctness, usefulness, or coverage.*

*Use the following criteria when assigning the score. **Intent satisfaction** evaluates whether the response identifies the user’s actual goal in the current turn and provides a useful answer or action for that goal.*

***Dialogue context usage** evaluates whether the response correctly incorporates relevant prior turns, carries over established constraints, resolves references such as “it” or “that” appropriately, and avoids being distracted by obsolete or unrelated context. **Instruction following** evaluates whether the response obeys explicit requirements about format, language, style, scope, tools, refusal behavior, or output length. **Factuality and reasoning** evaluates whether the response is factually correct, logically consistent, and supported by the information available in the conversation or by reliable reasoning. **Completeness** evaluates whether all important parts of the request are addressed, including edge cases, caveats, or requested explanations. **Clarity***

***and usefulness** evaluates whether the response is easy to understand, well organized, appropriately concise, and practically helpful to the user.*

*Apply the following scoring guideline. A score of **10** should be reserved for a response that fully satisfies the user’s intent, uses all necessary dialogue context correctly, follows every instruction, is factually sound, complete, and clear. A score of **9** indicates an excellent response with only negligible wording or presentation issues. Scores of **7–8** indicate a mostly correct and useful response that satisfies the main request but has minor omissions, mild ambiguity, limited explanation, or only partial use of relevant context. Scores of **5–6** indicate a partially useful response that addresses the general topic but misses important constraints, overlooks meaningful dialogue history, contains noticeable but non-fatal factual or reasoning errors, or leaves substantial parts of the task incomplete. Scores of **3–4** indicate a weak response that is only loosely related to the request, substantially incomplete, poorly grounded in the dialogue, or affected by major factual, reasoning, or instruction-following errors. Scores of **1–2** indicate a response that is irrelevant, contradicts the conversation, refuses without justification, is unsafe when safety is required, fails to answer the user, or is dominated by hallucinated content.*

*When several issues are present, choose the score that best reflects the overall usefulness of the response to the user. Penalize strongly for errors that would cause the user to take a wrong action, violate an explicit constraint, or misunderstand the task outcome. A response with a major factual error should not receive a score above 6, even if it is fluent. A response that ignores the current user request or relies on the wrong dialogue context should not receive a score above 4. A response that is correct but unnecessarily verbose, poorly organized, or missing minor details may still receive a high score if the user’s intent is satisfied.*

*Return only the final score.*

The judge model returns an integer rating from 1 to 10, which we normalize to the range  $[0, 1]$  as the true score  $w_{i,t}$  for evaluation metrics computation.

## 6.2 Overall Performance

As shown in Figure 4, SWRouter achieves the best overall performance across the two multi-turn dialogue datasets. Compared with the strongest single LLM, dolphin-2.9-llama3-8b, SWRouter improves the average **evaluation accuracy** from 24.63% to 40.89%, corresponding to an absolute gain of 16.26%. The improvement is consistent across both datasets, with gains of 14.29% on ShareGPT and 18.22% on MTBench.

Compared with Conv-ID Context, which preserves the original dialogue segmentation by conversation IDs, SWRouter further improves the average **evaluation accuracy** by 8.22%. This result suggests that exact ID-based partitioning is not necessarily the optimal context construction strategy for multi-turn routing. Instead, dynamically constructing semantic windows provides more useful routing inputs.

Moreover, ZOETER equipped with the similarity-window mechanism achieves 39.21% average **evaluation accuracy**, substantially outperforming Conv-ID Context. This indicates that the proposed similarity-window design is general and can improve different routing backbones by constructing better multi-turn contexts.

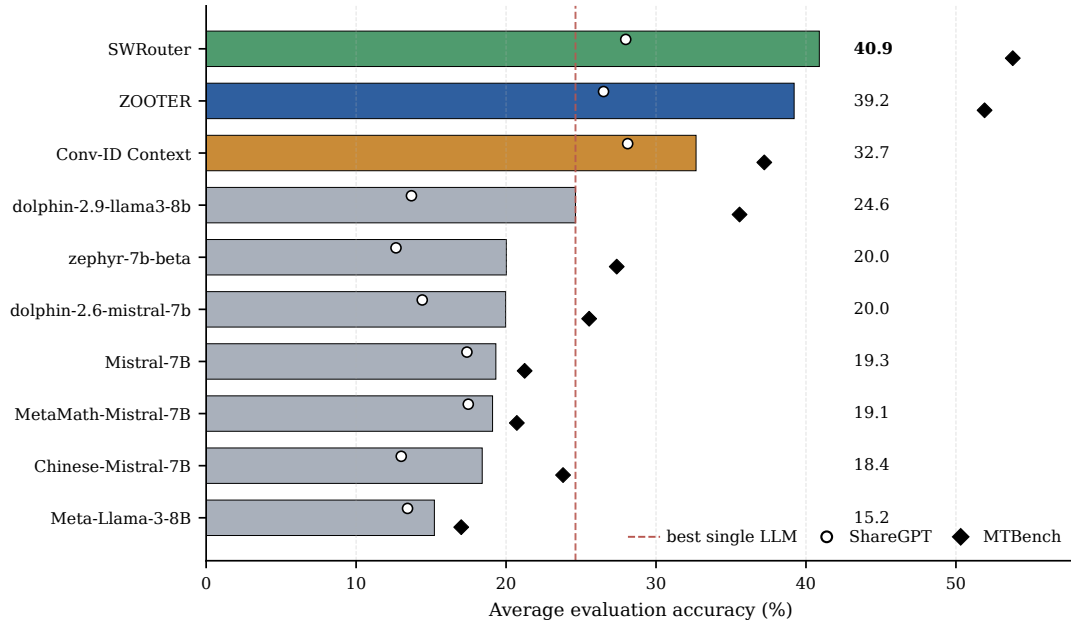


Fig. 4. Overall **evaluation accuracy** (%) on ShareGPT and MTBench. Horizontal bars rank all candidate single LLMs and routing methods by average accuracy, while markers show the corresponding ShareGPT and MTBench scores. The dashed line marks the strongest single LLM, making the routing gain of SWRouter visually explicit.

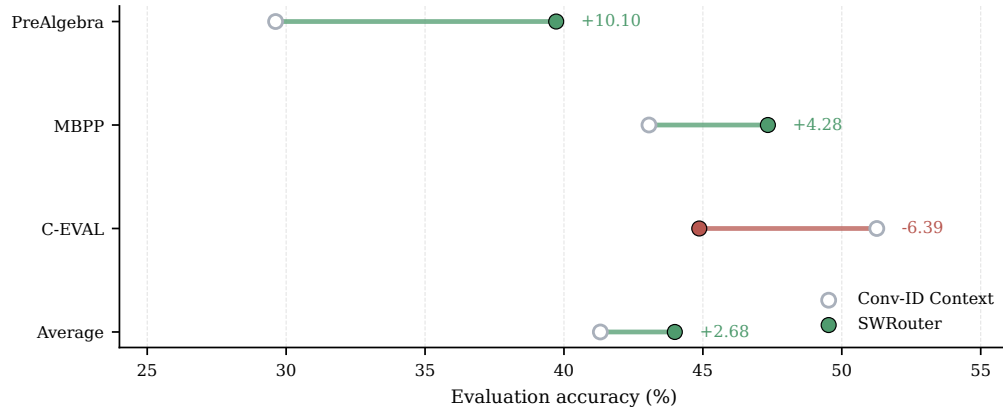


Fig. 5. **Evaluation accuracy** (%) of SWRouter and Conv-ID Context on out-of-distribution datasets. PreAlgebra, MBPP, and C-EVAL cover mathematical reasoning, code generation, and Chinese knowledge-intensive evaluation, respectively. The connected markers compare SWRouter with Conv-ID Context on each task, and the right-side annotations report the absolute gain or loss. This visualization emphasizes that SWRouter improves the average OOD result despite not winning on every individual task.



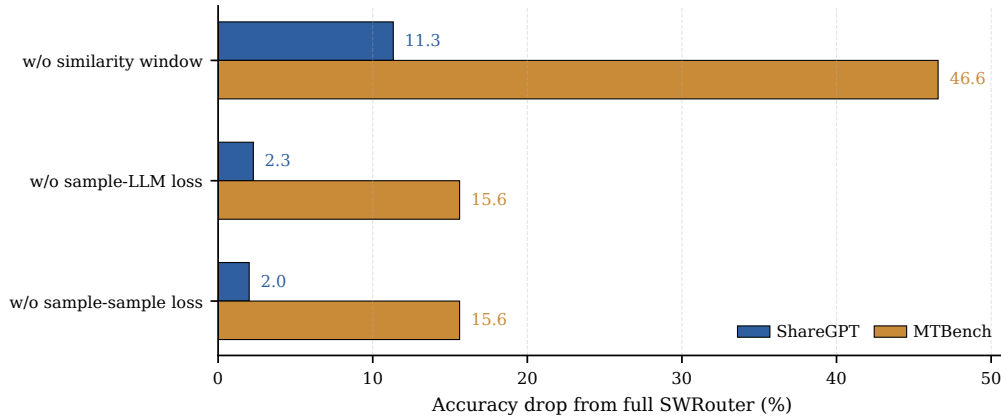


Fig. 6. Ablation study results measured by **evaluation accuracy** (%) of SWRouter . Each bar reports the accuracy drop relative to the full SWRouter model after removing one component. The full model obtains 27.98% on ShareGPT and 53.79% on MTBench. This drop-oriented view highlights that similarity-window construction is the dominant contributor, especially on MTBench, while both contrastive losses also provide consistent gains.

### 6.3 OOD Generalization

We further evaluate the generalization ability of SWRouter under out-of-distribution scenarios, including *PreAlgebra*, *MBPP*, and *C-EVAL*. These datasets cover mathematical reasoning, code generation, and Chinese knowledge-intensive evaluation, respectively, and are related to established mathematical and Chinese multitask evaluation settings [11, 25].

As shown in Figure 5, SWRouter achieves the best average OOD performance, reaching 43.99% and outperforming Conv-ID Context by 2.68%. In particular, SWRouter improves Conv-ID Context by 10.10% on *PreAlgebra* and 4.28% on *MBPP*. Although Conv-ID Context performs better on *C-EVAL*, SWRouter exhibits stronger average robustness across heterogeneous OOD tasks.

This result indicates that SWRouter does not simply fit ID-based routing labels. Instead, the learned selection strategy captures transferable routing patterns that generalize beyond the original dialogue distribution.

### 6.4 Performance Breakdown

We conduct ablation studies to quantify the contribution of each component in SWRouter . As shown in Figure 6, removing any component consistently degrades performance, showing that all components are necessary for effective routing.

**Similarity window.** Removing the similarity window causes the largest performance drop. **Evaluation Accuracy** decreases by 11.33% on ShareGPT and 46.57% on MTBench. This confirms that adaptive context construction is the most important component of SWRouter .

**Sample-LLM loss.** Removing  $L_{\text{sample-LLM}}$  reduces performance to 25.70% on ShareGPT and 38.17% on MTBench. This indicates that modeling the relationship between dialogue samples and candidate LLMs is critical for accurate routing.

**Sample-sample loss.** Removing  $L_{\text{sample-sample}}$  also significantly hurts performance, especially on MTBench. This shows that sample-level representation alignment helps construct a more structured routing space.

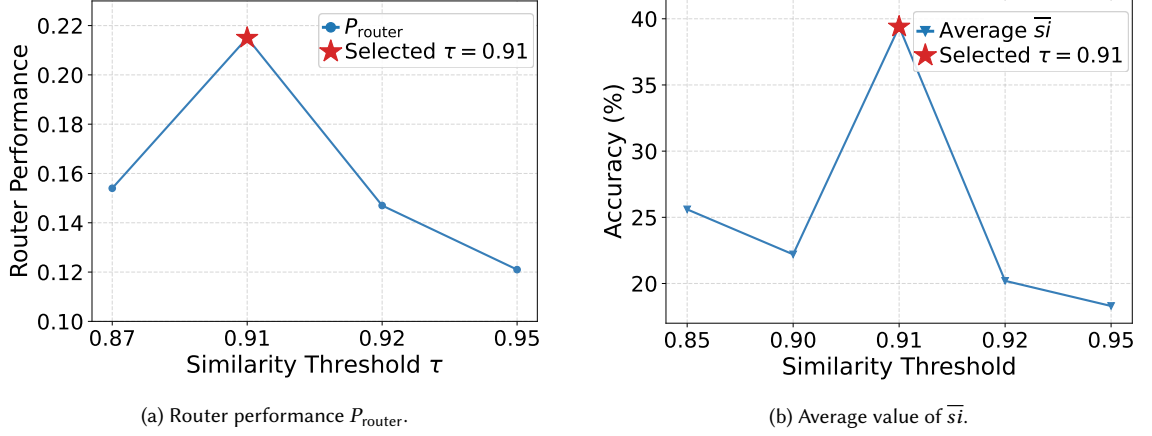


Fig. 7. Effect of the similarity threshold  $\tau$  on router performance  $P_{\text{router}}$  and the average value of  $\bar{s}_i$ . The left subfigure shows how strongly the router improves over the average candidate-model score under each threshold, while the right subfigure reports the resulting end-to-end routed response quality. Together, the curves show that threshold selection affects both model-selection effectiveness and absolute response quality, with the best operating region concentrated around  $\tau = 0.91$ .

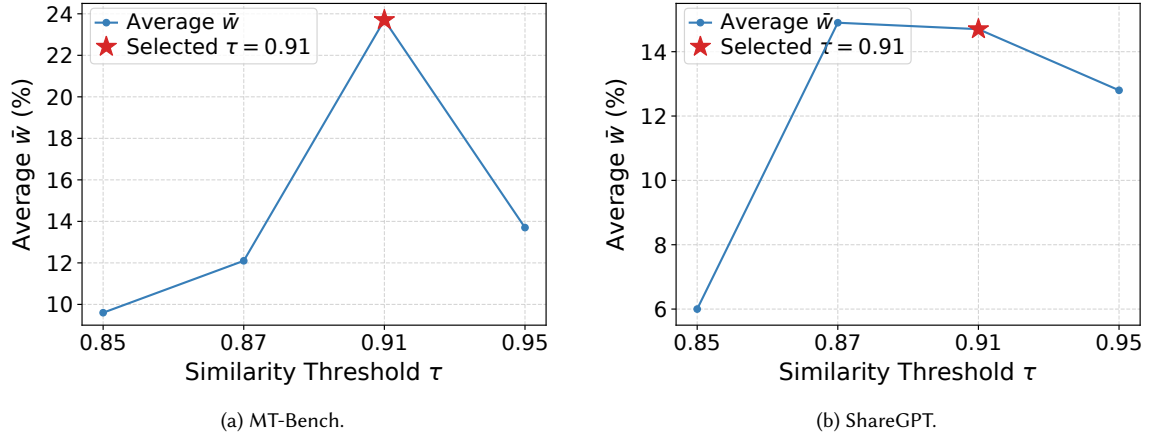


Fig. 8. Effect of the similarity threshold  $\tau$  on the average value of  $\tilde{w}$  for MT-Bench and ShareGPT. Since  $\tilde{w}$  averages the true scores of all candidate LLMs under the constructed prompts, these curves isolate the prompt-construction quality from the router’s model-selection behavior. The comparison shows how stricter or looser window partitioning changes the amount of useful context retained for the candidate model pool.

**Actual score.** We further observe that proxy routing scores can substantially overestimate the final end-to-end performance if prompt construction is ignored. This highlights the necessity of evaluating actual generated responses rather than relying only on intermediate routing objectives.

## 6.5 Sensitivity Analysis

We analyze the sensitivity of SWRouter to the similarity threshold  $\tau$ .

**Effect of Similarity Threshold  $\tau$ .** We conduct a sensitivity study on the similarity threshold  $\tau$  with respect to three key performance metrics: the average value of  $\bar{s}i$ , the average value of  $\bar{w}$ , and router performance  $P_{\text{router}}$ .

**Average Value of  $\bar{s}i$**  ( $\bar{s}i = \frac{1}{n} \sum_{i=1}^n s_i$ ). Figure 7(b) shows the effect of  $\tau$  on  $\bar{s}i$ . The similarity-window construction achieves its best performance at  $\tau = 0.91$ , reaching 40.89% accuracy and outperforming the Conv-ID Context baseline of 32.67%. When  $\tau$  moves away from this value, the performance decreases noticeably. For example, the accuracy drops to 25.6% at  $\tau = 0.85$ , 22.2% at  $\tau = 0.90$ , 20.2% at  $\tau = 0.92$ , and 18.3% at  $\tau = 0.95$ . These results indicate that  $\bar{s}i$  is highly sensitive to the similarity threshold, and that  $\tau = 0.91$  provides the most effective similarity window.

**Average Value of  $\bar{w}$ .** Figure 8 presents the average value of  $\bar{w}$  under different similarity thresholds  $\tau$  on MT-Bench and ShareGPT. The results show that  $\bar{w}$  is strongly affected by the similarity threshold. On MT-Bench,  $\bar{w}$  increases from 9.60% at  $\tau = 0.85$  to 12.10% at  $\tau = 0.87$ , and further reaches the maximum value of 23.70% at  $\tau = 0.91$ . However, when the threshold is increased to  $\tau = 0.95$ ,  $\bar{w}$  drops sharply to 13.70%, suggesting that an overly strict similarity constraint can substantially reduce the retained high-quality samples.

A similar trend can also be observed on ShareGPT. The average value of  $\bar{w}$  is 6.00% at  $\tau = 0.85$ , increases significantly to 14.90% at  $\tau = 0.87$ , and remains at a comparable level of 14.70% when  $\tau = 0.91$ . It then decreases to 12.80% at  $\tau = 0.95$ . These results suggest that ShareGPT is slightly more stable than MT-Bench around the optimal region, but its performance is still affected by threshold selection.

Overall, the high-quality region is mainly concentrated around  $\tau = 0.91$ . MT-Bench achieves its best result at  $\tau = 0.91$ , while ShareGPT maintains competitive performance in the range of  $\tau \in [0.87, 0.91]$ . Considering both datasets,  $\tau = 0.91$  provides a stable trade-off between sample quality and coverage, and we therefore adopt it as the default threshold.

**Router Performance ( $P_{\text{router}}$ ).** Figure 7(a) illustrates the effect of the similarity threshold  $\tau$  on router performance  $P_{\text{router}}$ . Overall,  $P_{\text{router}}$  is influenced by the choice of  $\tau$ , although its variation is less pronounced than that of  $\bar{w}$ . Specifically,  $P_{\text{router}}$  is 15.4% at  $\tau = 0.87$ , increases to its peak of 21.5% at  $\tau = 0.91$ , and then decreases to 14.7% at  $\tau = 0.92$  and 12.1% at  $\tau = 0.95$ .

These results indicate that router performance still depends on the similarity threshold, but the overall fluctuation is relatively moderate. Compared with  $\bar{w}$ ,  $P_{\text{router}}$  is less sensitive to changes in  $\tau$ . Nevertheless, the best router performance is still achieved at  $\tau = 0.91$ , further supporting our choice of  $\tau = 0.91$  as the default threshold.

## 6.6 Robustness Analysis

**6.6.1 Extension: Top-K Retrieval for Long-Range Dependency Recovery.** As an extension to the core similarity-based windowing approach, we investigate whether augmenting it with a lightweight Top-K retrieval mechanism can further recover semantically related queries that are separated by unrelated turns.

We simulate a scenario where a user issues a query  $a$ , then several unrelated queries, and finally a related query  $b$ . For each  $b$ , similarity-based Top-K retrieval is performed over the dialogue history to test whether  $a$  can be recalled.

Table 2. Recall performance of similarity-based retrieval under different levels of semantic interference. Inserted queries denote unrelated turns placed between two semantically related user requests; Top-K recall measures whether the retrieval module can recover the earlier related turn from the dialogue history.

| Inserted Queries | Top-1 Recall | Top-3 Recall | Top-5 Recall |
|------------------|--------------|--------------|--------------|
| 1                | 79.43        | 100.00       | 100.00       |
| 3                | 65.16        | 91.31        | 100.00       |
| 5                | 59.84        | 77.84        | 94.15        |

We also test a more challenging case where 20 unrelated queries are inserted between  $a$  and  $b$ . The results are shown below.

Table 3. Retrieval robustness under the high-difficulty setting with 20 distractor turns. This setting stresses long-range dependency recovery by separating two related requests with many unrelated turns, and the reported recall indicates whether Top- $K$  retrieval can still recover the relevant earlier query.

| Inserted Queries | Top-K | Recall (%) | Difficulty |
|------------------|-------|------------|------------|
| 20               | 5     | 66.13      | Hard       |

As shown in Tables 2 and 3, similarity-based retrieval substantially improves robustness under semantic drift. Top-3 retrieval provides an effective trade-off between recall and efficiency. Even under high-difficulty conditions with 20 distractors, Top-5 recall remains above 66%, indicating strong resilience. These results demonstrate that augmenting local similarity-based segmentation with retrieval mechanisms can further enhance context reconstruction in realistic multi-turn dialogue scenarios.

**6.6.2 Semantic Coherence in Long Dialogues.** To further analyze the potential semantic drift as the dialogue length increases, we conducted a similarity analysis over multi-turn dialogues on the *ShareGPT* and *MT-Bench* datasets. Specifically, we computed the cosine similarity between the first and last user turns within each dialogue window under different minimum round sizes (20, 25, and 30).

As shown in Table 4, in *ShareGPT*, over 70% of long-range pairs maintain a cosine similarity above 0.91, and the average similarity remains above 0.92, suggesting strong semantic consistency even across extended dialogues. In *MT-Bench*, the average similarity is slightly lower but still indicates limited semantic drift. Extremely low similarity cases ( $<0.80$ ) are rare, implying that our similarity-based segmentation remains semantically coherent over long dialogue windows.

## 6.7 Detailed Overhead Analysis

We conduct a comprehensive analysis of the computational overhead introduced by SWRouter from three aspects: inference latency, training cost, and token generation cost.

**6.7.1 Latency Analysis.** As shown in Table 5, the similarity calculation in SWRouter accounts for less than 1% (approximately 2.2‰) of the total inference time, indicating high computational efficiency. The dominant cost comes from LLM inference, which is consistent with all routing-based systems.

**6.7.2 Training Cost Analysis.** The training process of SWRouter consists of four steps:

- (1) **Dataset window partitioning:** We use a lightweight encoder (microsoft/mdeberta-v3-base) with approximately 86M backbone parameters, which is much smaller than the 7B/8B candidate LLMs, making this step’s cost negligible.
- (2) **Candidate response generation:** We use the seven small models mentioned in the paper (Mistral-7B, MetaMath-Mistral-7B, zephyr-7b-beta, Chinese-Mistral-7B, dolphin-2.6-mistral-7b, Meta-Llama-3-8B, dolphin-2.9-llama3-8b) to generate candidate answers for each query.
- (3) **Judge-model scoring:** We use GPT for scoring. Since the output is just a single numerical score, this step does not significantly increase memory usage.

Table 4. Cosine similarity between the first and last user turns across dialogues of varying lengths. For each dataset, we group dialogues by minimum round size and report the proportion of long-range turn pairs above or below the default similarity threshold  $\tau = 0.91$ , together with the maximum, minimum, and average similarity. The table characterizes how quickly semantic drift appears as dialogue length increases.

| Dataset  | Min Group Size | Total Pairs | $\geq 0.91$ (%) | $< 0.91$ (%) | Max    | Min    | Avg    |
|----------|----------------|-------------|-----------------|--------------|--------|--------|--------|
| ShareGPT | 20             | 19          | 73.68           | 26.32        | 0.9922 | 0.6774 | 0.9283 |
|          | 25             | 13          | 76.92           | 23.08        | 0.9922 | 0.6774 | 0.9211 |
|          | 30             | 7           | 71.43           | 28.57        | 0.9874 | 0.8035 | 0.9247 |
| MT-Bench | 20             | 7           | 57.14           | 42.86        | 0.9620 | 0.7961 | 0.8989 |
|          | 25             | 6           | 50.00           | 50.00        | 0.9620 | 0.7961 | 0.8928 |
|          | 30             | 4           | 25.00           | 75.00        | 0.9426 | 0.7961 | 0.8603 |

Table 5. Time consumption of SWRouter during inference. The table decomposes latency into similarity calculation for window construction, router selection, and downstream LLM inference, showing that the additional cost introduced by similarity-window routing is small relative to candidate-model generation.

| Time     | Similarity Calculation | Router Selection | LLM Inference |
|----------|------------------------|------------------|---------------|
| SWRouter | 50.6s                  | 16min32s         | >6h           |

- (4) **Router training:** The similarity-window encoder introduces limited overhead: microsoft/mdeberta-v3-base has approximately 86M backbone parameters, which is much smaller than the 7B/8B candidate LLMs. Moreover, the encoder is used only for lightweight similarity computation, while the dominant cost remains candidate LLM inference. Router training is performed only once before deployment, making it a one-time overhead.

**6.7.3 Token Generation Cost.** To evaluate the efficiency of model selection, we compare the token-level costs between SWRouter and the Conv-ID Context method. Due to the unavailability of precise pricing information for small-sized models such as 7B and 8B, we adopt token count as a unified proxy for cost estimation.

Table 6. Overall evaluation accuracy and token cost comparison. Evaluation accuracy measures response quality after routing, while token cost counts the generated tokens used by each routing strategy as a model-agnostic proxy for serving cost. The comparison highlights the effectiveness–cost tradeoff between Conv-ID Context, SimWindow + ZOOPER, and SWRouter.

| Router             | Evaluation Acc. $\uparrow$ | Token Cost $\downarrow$ |
|--------------------|----------------------------|-------------------------|
| Conv-ID Context    | 32.67                      | 2834                    |
| SimWindow + ZOOPER | 39.20                      | 3405                    |
| SWRouter           | <b>40.89</b>               | 3477                    |

As shown in Table 6, SWRouter incurs approximately  $1.23\times$  the token cost of Conv-ID Context while achieving an 8.22% absolute improvement in evaluation accuracy. Compared to ZOOPER, SWRouter requires only  $1.02\times$  the token cost, yet delivers a further 1.69% accuracy gain. This result suggests that SWRouter achieves a reasonable trade-off between performance and efficiency, despite lacking access to ground-truth optimal selections.

**6.7.4 Additional Analysis.** We further analyze the experimental results from the following four aspects:

(1) **Prompt Construction Impact.** The similar performance patterns across models at different similarity thresholds suggest that prompt construction, rather than model architecture, is the primary performance driver.  $\tau = 0.91$  performs best among tested thresholds; performance is relatively strong in the neighborhood around 0.91.

(2) **Router-Score Correlation.** Analysis reveals a weak negative correlation (Pearson  $R = -0.2301$ ) between router performance and true scores, indicating that higher true scores do not necessarily translate to better routing performance. This underscores the importance of decoupled evaluation metrics.

(3) **Prompt Construction Dominance.**  $\tau = 0.91$  performs best among tested thresholds; performance is relatively strong in the neighborhood around 0.91, emphasizing that prompt construction quality outweighs router performance in multi-turn dialogue systems.

(4) **Exact Partitioning is Not Necessarily Optimal.** This can be compared from three aspects: (i) On **Evaluation Accuracy** ( $\bar{s}_i$ ), when  $\tau = 0.91$  with 40.89%, it is 8.22% higher than Conv-ID Context (32.67%), proving that the router model trained with exact partitioning does not perform as well as SWRouter. (ii) On **Construction Accuracy** ( $\bar{w}$ ), when the similarity threshold  $\tau$  is 0.90, the average score of 7 individual models significantly outperforms the prompt constructed by Conv-ID Context, indicating that in multi-turn dialogue scenarios, strictly accurate context partitioning cannot be considered the optimal context segmentation method. (iii) On **Router Performance** ( $P_{\text{router}}$ ), the average performance for both  $\tau = 0.85$  and  $\tau = 0.91$  surpass Conv-ID Context, meaning that the prompt constructed by Conv-ID Context also fails to train the optimal router performance.

Overall, SWRouter introduces limited additional computational overhead while substantially improving routing performance, demonstrating a favorable effectiveness–efficiency tradeoff for multi-turn LLM routing.

## 7 Related Work

Large language models have become increasingly heterogeneous in capability, cost, latency, and domain specialization. This heterogeneity has motivated systems that use multiple LLMs rather than committing to a single model for all inputs. Existing work can be broadly grouped into LLM ensembling and cascading, single-turn LLM routing, and context-aware routing for multi-turn dialogue.

### 7.1 LLM Ensembling and Cascading

LLM ensembling aims to improve response quality by consulting multiple models or multiple generations before producing the final answer. A simple and widely used strategy is voting or self-consistency, where multiple reasoning paths or model outputs are aggregated to reduce variance and improve reliability [26, 45]. More structured ensemble methods go beyond voting: LLM-Blender [23] first ranks candidate outputs with PairRanker and then synthesizes a final response with GenFuser, showing that complementary model outputs can be combined into a stronger answer.

Another line of work focuses on cascades, where models are queried sequentially according to estimated difficulty, confidence, or cost. FrugalGPT [8] studies cost-effective model selection through cascaded calls, while language-model cascades further explore uncertainty-aware and token-level routing policies [18, 47]. Online cascade learning extends this idea to streaming or adaptive inference settings [34]. These methods reduce cost compared with invoking all available models, but they may still require multiple model calls for a single user request. In contrast, SWRouter follows the routing paradigm: it aims to choose one suitable candidate LLM after constructing the multi-turn prompt, thereby avoiding repeated generation from many models at inference time.

### 7.2 Single-turn LLM Routing

LLM routing selects the most suitable model for a query without necessarily calling all candidate LLMs. Early routing studies often formulate the problem as correctness prediction or reward prediction for each candidate model. Shnitzer et al. [40] construct benchmark datasets for LLM routing and train model-specific binary classifiers to estimate whether

each model will answer correctly. ZOOTOER [33] aligns a router with reward-model supervision and shows that learned routers can outperform static model selection. Cost-aware routing methods such as C2MAB-V [14] further incorporate online decision making and reward signals to balance model quality and inference cost.

Recent work also studies representation-based routing. LoraRetriever [48] routes inputs by predicting task identity and retrieving suitable LoRA modules, while Srivatsa et al. [41] analyze classifier-based and clustering-based routing strategies. RouterDC [9] introduces a dual contrastive learning objective that models both query-model relations and query-query relations, making it a strong single-turn router backbone. These methods provide useful supervision and representation-learning tools for model selection, and SWRouter builds on this direction. However, they generally assume that each routing instance is already a complete prompt. This assumption is reasonable for single-turn benchmarks, but it becomes fragile in multi-turn dialogue because the routing input itself depends on how historical context is selected and assembled.

### 7.3 Context Construction for Multi-turn Dialogue

Multi-turn dialogue introduces a context-construction problem before routing can even begin. A conversational request may depend on previous constraints, definitions, or preferences, but the full dialogue history may also contain obsolete or off-topic content. Directly concatenating all previous turns can therefore increase prompt length and introduce irrelevant information, whereas using only the current turn can omit essential context. Long-context benchmarks such as MT-Bench-101 [6] and real dialogue data such as ShareGPT [38] highlight that multi-turn tasks often require tracking user intent across turns rather than treating each prompt independently.

Several adjacent research areas offer partial solutions. Retrieval-based context construction can recover long-range dependencies by searching dialogue history for semantically related turns. Sequence segmentation and similarity-based partitioning, rooted in broader sequence analysis and string-processing ideas [1, 7, 19], provide a way to divide a conversation into locally coherent windows. Neural encoders such as DeBERTa [20] can then map turns into dense representations for semantic matching. Nevertheless, these techniques are usually studied as prompt-construction or retrieval components rather than as part of a routing evaluation framework. SWRouter integrates similarity-based window construction with router learning so that the selected model is conditioned on a prompt that is explicitly designed for multi-turn relevance.

### 7.4 Evaluation of Routing Under Constructed Prompts

Most routing evaluations focus on whether the router selects a high-scoring model for a given input. This is appropriate when the input prompt is fixed and complete, but it can be misleading when prompt construction varies. In multi-turn dialogue, a poorly constructed prompt can lower the absolute scores of all candidate LLMs even if their relative ranking remains unchanged. Conversely, a better prompt can improve all candidate responses while leaving router-selection difficulty similar. Standard aggregate metrics such as benchmark accuracy [11, 21, 25] or reward-model score therefore conflate two factors: whether the constructed prompt preserves enough information, and whether the router chooses the best model under that prompt.

The key difference between SWRouter and prior routing work is this explicit separation. Instead of evaluating only the selected model’s final score, SWRouter reports evaluation accuracy, construction accuracy, and router performance. This decoupled view makes it possible to determine whether improvements come from better prompt construction, better model selection, or both. As a result, SWRouter extends single-turn LLM routing to a setting where context segmentation is a first-class part of the routing problem rather than a preprocessing detail.



## 8 Conclusion

This paper presents SWRouter, a similarity-contractive window router that addresses two fundamental challenges in multi-turn LLM routing: information loss and information confusion caused by improper context segmentation, and attribution bias in router evaluation that conflates prompt construction quality with routing effectiveness.

To tackle these challenges, SWRouter introduces a similarity-based window partitioning mechanism that dynamically constructs semantically coherent prompts from dialogue history, and a decoupled evaluation framework comprising three complementary metrics: evaluation accuracy ( $\bar{s}_i$ ), construction accuracy ( $\bar{w}$ ), and router performance ( $P_{\text{router}}$ ). Extensive experiments on ShareGPT and MTBench demonstrate that SWRouter achieves **16.26%** and **8.22%** gains over the best individual LLM and the Conv-ID Context baseline, respectively. On out-of-distribution tasks (PreAlgebra, MBPP, and C-EVAL), SWRouter further outperforms Conv-ID Context by **2.68%** on average, demonstrating strong generalization. Decoupled metric analysis confirms that the gains are primarily driven by improved construction accuracy, while  $P_{\text{router}} > 1$  consistently validates effective model selection. These results highlight that multi-turn LLM routing requires a joint design of context construction and evaluation, rather than a direct extension of single-turn routing methods.

Despite these advances, the current framework is evaluated on 7B/8B scale models; its behavior under larger or proprietary LLMs remains to be studied. Future work may extend the routing framework to heterogeneous model pools spanning different scales and explore its applicability to proprietary LLMs.

## References

- [1] Alfred V. Aho and Jeffrey D. Ullman. 1972. *The Theory of Parsing, Translation, and Compiling*. Vol. 1. Prentice-Hall, Englewood Cliffs, NJ.
- [2] American Psychological Association. 1983. *Publications Manual*. American Psychological Association, Washington, DC.
- [3] Rie Kubota Ando and Tong Zhang. 2005. A Framework for Learning Predictive Structures from Multiple Tasks and Unlabeled Data. *Journal of Machine Learning Research* 6 (2005), 1817–1853. <http://jmlr.org/papers/v6/ando05a.html>
- [4] Galen Andrew and Jianfeng Gao. 2007. Scalable Training of L1-Regularized Log-Linear Models. In *Proceedings of the 24th International Conference on Machine Learning*. ACM, Corvallis, OR, USA, 33–40. <https://doi.org/10.1145/1273496.1273501>
- [5] Anthropic. 2024. Claude. <https://docs.anthropic.com/en/docs/welcome>.
- [6] Ge Bai, Jie Liu, Xingyuan Bu, Yancheng He, Jiaheng Liu, Zhanhui Zhou, Zhuoran Lin, Wenbo Su, Tiezheng Ge, Bo Zheng, and Wanli Ouyang. 2024. MT-Bench-101: A Fine-Grained Benchmark for Evaluating Large Language Models in Multi-Turn Dialogues. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Bangkok, Thailand, 7421–7454. <https://doi.org/10.18653/V1/2024.ACL-LONG.401>
- [7] Ashok K. Chandra, Dexter Kozen, and Larry J. Stockmeyer. 1981. Alternation. *J. ACM* 28, 1 (1981), 114–133. <https://doi.org/10.1145/322234.322243>
- [8] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *CoRR* abs/2305.05176 (2023). <https://doi.org/10.48550/ARXIV.2305.05176> arXiv:2305.05176
- [9] Shuhao Chen, Weisen Jiang, Baijiong Lin, James T. Kwok, and Yu Zhang. 2024. RouterDC: Query-Based Router by Dual Contrastive Learning for Assembling Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Vancouver, BC, Canada, 66305–66328. [https://proceedings.neurips.cc/paper\\_files/paper/2024/hash/7a641b8ec86162fc875fb9f6456a542f-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2024/hash/7a641b8ec86162fc875fb9f6456a542f-Abstract-Conference.html)
- [10] Xinran Chen, Yuchen Li, Hengyi Cai, Zhuoran Ma, Xuanang Chen, Haoyi Xiong, Shuaiqiang Wang, Ben He, Le Sun, and Dawei Yin. 2025. Multi-agent proactive information seeking with adaptive llm orchestration for non-factoid question answering. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 4341–4352.
- [11] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *CoRR* abs/2110.14168 (2021). arXiv:2110.14168 <https://arxiv.org/abs/2110.14168>
- [12] Cognitive Computations. 2024. cognitivecomputations/dolphin-2.6-mistral-7b. <https://huggingface.co/cognitivecomputations/dolphin-2.6-mistral-7b>.
- [13] Cognitive Computations. 2024. cognitivecomputations/dolphin-2.9-llama3-8b. <https://huggingface.co/cognitivecomputations/dolphin-2.9-llama3-8b>.
- [14] Xiangxiang Dai, Jin Li, Xutong Liu, Anqi Yu, and John C. S. Lui. 2024. Cost-Effective Online Multi-LLM Selection with Versatile Reward Models. *CoRR* abs/2405.16587 (2024). <https://doi.org/10.48550/ARXIV.2405.16587> arXiv:2405.16587
- [15] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. *CoRR* abs/2412.19437 (2024). <https://doi.org/10.48550/ARXIV.2412.19437> arXiv:2412.19437



- [16] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A Framework for Few-Shot Language Model Evaluation. <https://zenodo.org/records/10256836>. <https://doi.org/10.5281/zenodo.10256836>
- [17] Gemini Team. 2023. Gemini: A Family of Highly Capable Multimodal Models. *CoRR* abs/2312.11805 (2023). <https://doi.org/10.48550/ARXIV.2312.11805> arXiv:2312.11805
- [18] Neha Gupta, Harikrishna Narasimhan, Wittawat Jitkrittum, Ankit Singh Rawat, Aditya Krishna Menon, and Sanjiv Kumar. 2024. Language Model Cascades: Token-Level Uncertainty and Beyond. *CoRR* abs/2404.10136 (2024). <https://doi.org/10.48550/ARXIV.2404.10136> arXiv:2404.10136
- [19] Dan Gusfield. 1997. *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, Cambridge, UK. <https://doi.org/10.1017/CBO9780511574931>
- [20] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. DeBERTa: Decoding-Enhanced BERT with Disentangled Attention. In *The Ninth International Conference on Learning Representations*. OpenReview.net, Virtual Event, Austria. <https://openreview.net/forum?id=XPZlaotutsD>
- [21] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *The Ninth International Conference on Learning Representations*. OpenReview.net, Virtual Event, Austria. <https://openreview.net/forum?id=d7KBjml3GmQ>
- [22] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. Mistral 7B. *CoRR* abs/2310.06825 (2023). <https://doi.org/10.48550/ARXIV.2310.06825> arXiv:2310.06825
- [23] Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Toronto, Canada, 14165–14178. <https://doi.org/10.18653/V1/2023.ACL-LONG.792>
- [24] Solomon Kullback and Richard A. Leibler. 1951. On Information and Sufficiency. *The Annals of Mathematical Statistics* 22, 1 (1951), 79–86. <https://doi.org/10.1214/aoms/117729694>
- [25] Haonan Li, Yixuan Zhang, Fajri Koto, Yifei Yang, Hai Zhao, Yeyun Gong, Nan Duan, and Timothy Baldwin. 2024. CMMLU: Measuring Massive Multitask Language Understanding in Chinese. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 11260–11285. <https://doi.org/10.18653/V1/2024.FINDINGS-ACL.671>
- [26] Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. 2024. More Agents Is All You Need. *CoRR* abs/2402.05120 (2024). <https://doi.org/10.48550/ARXIV.2402.05120> arXiv:2402.05120
- [27] Yuchen Li, Hengyi Cai, Rui Kong, Xinran Chen, Jiamin Chen, Jun Yang, Haojie Zhang, Jiayi Li, Jiayi Wu, Yiqun Chen, et al. 2025. Towards ai search paradigm. *arXiv preprint arXiv:2506.17188* (2025).
- [28] Yuchen Li, Jiamin Chen, Xinran Chen, Zhiyu Li, Haojie Zhang, Rui Kong, Jiayi Li, Xinyu Ma, Hengyi Cai, Lixin Su, et al. 2026. Retain to Refine: Adaptive Online Question Answering via Query Routing and Long-Short Memory. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 2312–2322.
- [29] Yuchen Li, Rui Kong, Xinran Chen, Chengzhe Zhang, Jiamin Chen, Cheng Deng, Xinyu Ma, Haojie Zhang, Tianhao Peng, Hengyi Cai, et al. 2026. Probe-and-fetch: Dynamic KV cache pruning for accelerated long-context inference in web-scale AI search. In *Proceedings of the ACM Web Conference 2026*. 8127–8137.
- [30] Yuchen Li, Rui Kong, Zhonghao Lyu, Qiyang Li, Xinran Chen, Hengyi Cai, Lingyong Yan, Shuaiqiang Wang, Jiashu Zhao, Guangxu Zhu, et al. 2026. Flexspec: Frozen drafts meet evolving targets in edge-cloud collaborative llm speculative decoding. *IEEE Transactions on Mobile Computing* (2026).
- [31] Llama Team. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024). <https://doi.org/10.48550/ARXIV.2407.21783> arXiv:2407.21783
- [32] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*. OpenReview.net, New Orleans, LA, USA. <https://openreview.net/forum?id=Bkg6RiCqY7>
- [33] Keming Lu, Hongyi Yuan, Runji Lin, Junyang Lin, Zheng Yuan, Chang Zhou, and Jingren Zhou. 2024. Routing to the Expert: Efficient Reward-Guided Ensemble of Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Mexico City, Mexico, 1964–1974. <https://doi.org/10.18653/V1/2024.NAACL-LONG.109>
- [34] Lunyiu Nie, Zhimin Ding, Erdong Hu, Christopher M. Jermaine, and Swarat Chaudhuri. 2024. Online Cascade Learning for Efficient Inference over Streams. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, Vienna, Austria, 38071–38090. <https://proceedings.mlr.press/v235/nie24a.html>
- [35] OpenAI. 2022. ChatGPT: Optimizing Language Models for Dialogue. <https://openai.com/blog/chatgpt>.
- [36] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., New Orleans, LA, USA, 53728–53741. [https://proceedings.neurips.cc/paper\\_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce7-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce7-Abstract-Conference.html)
- [37] Mohammad Sadegh Rasooli and Joel R. Tetreault. 2015. Yara Parser: A Fast and Accurate Dependency Parser. *CoRR* abs/1503.06733 (2015). arXiv:1503.06733 <http://arxiv.org/abs/1503.06733>
- [38] Philipp Schmid. 2023. ShareGPT 90k Raw Dataset. [https://huggingface.co/datasets/philschmid/sharegpt-raw/tree/main/sharegpt\\_90k\\_raw\\_dataset](https://huggingface.co/datasets/philschmid/sharegpt-raw/tree/main/sharegpt_90k_raw_dataset).

- [39] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *CoRR* abs/2402.03300 (2024). <https://doi.org/10.48550/ARXIV.2402.03300> arXiv:2402.03300
- [40] Tal Shnitzer, Anthony Ou, Mirian Silva, Kate Soule, Yuekai Sun, Justin Solomon, Neil Thompson, and Mikhail Yurochkin. 2023. Large Language Model Routing with Benchmark Datasets. *CoRR* abs/2309.15789 (2023). <https://doi.org/10.48550/ARXIV.2309.15789> arXiv:2309.15789
- [41] KV Aditya Srivatsa, Kaushal Kumar Maurya, and Ekaterina Kochmar. 2024. Harnessing the Power of Multiple Minds: Lessons Learned from LLM Routing. *CoRR* abs/2405.00467 (2024). <https://doi.org/10.48550/ARXIV.2405.00467> arXiv:2405.00467
- [42] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR* abs/2302.13971 (2023). <https://doi.org/10.48550/ARXIV.2302.13971> arXiv:2302.13971
- [43] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruiti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR* abs/2307.09288 (2023). <https://doi.org/10.48550/ARXIV.2307.09288> arXiv:2307.09288
- [44] Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro von Werra, Clémentine Fourrier, Nathan Habib, Nathan Sarrazin, Omar Sanseviero, Alexander M. Rush, and Thomas Wolf. 2023. Zephyr: Direct Distillation of LM Alignment. *CoRR* abs/2310.16944 (2023). <https://doi.org/10.48550/ARXIV.2310.16944> arXiv:2310.16944
- [45] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*. OpenReview.net, Kigali, Rwanda. <https://openreview.net/forum?id=1PL1NIMMrw>
- [46] Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2024. MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models. In *The Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria. <https://openreview.net/forum?id=N8N0hgNDRt>
- [47] Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. 2024. Large Language Model Cascades with Mixture of Thought Representations for Cost-Efficient Reasoning. In *The Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria. <https://openreview.net/forum?id=6okaSfANzh>
- [48] Ziyu Zhao, Leilei Gan, Guoyin Wang, Wangchunshu Zhou, Hongxia Yang, Kun Kuang, and Fei Wu. 2024. LoraRetriever: Input-Aware LoRA Retrieval and Composition for Mixed Tasks in the Wild. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 4447–4462. <https://doi.org/10.18653/V1/2024.FINDINGS-ACL.263>
- [49] Chen Zhou and Yuqi Bai. 2024. Chinese-Mistral: An Efficient and Effective Chinese Large Language Model. <https://github.com/THU-ESIS/Chinese-Mistral>.